Implementing Multithreading in the Parking System Server

ICT 4315: Object Oriented Methods and Programming II

for

Master of Science

Information Technology

Kalika Browder

University of Denver College of Professional Studies

May 31, 2026

Faculty: Nathan Braun, MBA
Director: Cathie Wilson, M.S.
Dean: Michael J. McGuire, MLS

Design

The central goal of the new and updated version of my Parking System for this assignment was to transform the parking server from a sequential, one-client-at-a-time design into a concurrent server capable of handling multiple simultaneous connections. The implementation follows the assignment's suggested progression: the client-handling logic was first extracted into its own ClientHandler class implementing Runnable, and then the server was modified to submit each new connection to a thread pool rather than blocking on it directly.

The threading model chosen is a fixed-size thread pool created with Executors.newFixedThreadPool(10). When the server's accept() call returns a new Socket, the server immediately wraps it in a ClientHandler instance and calls threadPool.submit(handler). Control returns to the main loop in microseconds, and the actual work (i.e. reading the JSON request, executing the command, writing the JSON response) happens concurrently on a pool worker thread. The server's main loop is therefore never blocked waiting for a client to finish. Instead, it can accept the next connection instantly.

Each ClientHandler logs a timestamp at construction and measures elapsed milliseconds before closing the socket, printing both the thread name and the handling duration. This timing instrumentation makes it straightforward to observe, in the server's console output, that multiple handlers are running simultaneously on distinct threads named pool-1-thread-N.

Thread safety in the shared ParkingOffice was addressed by replacing all plain ArrayList collections with Collections.synchronizedList(new ArrayList<>()). Because every public mutation

method (addCustomer, addCar, addLot, addCharge) acquires the intrinsic lock on its list before modifying it, concurrent registrations from different handler threads cannot interleave in a way that corrupts the data. The TransactionManager and PermitManager singletons held by ParkingOffice were similarly reviewed, and their internal state changes are also guarded by synchronized collections now.

Challenges

In this assignment, the most conceptually significant challenge was identifying all the shared mutable states that could be accessed simultaneously by multiple handler threads. In a single-threaded server this is a non-issue, but as soon as two ClientHandlerthreads execute office.addCustomer() concurrently, both threads touch the same ArrayList. Without synchronization the ArrayListcan end up in an inconsistent internal state, which is a scenario that manifests as data loss or ConcurrentModificationException at runtime rather than a compile-time error, making it easy to miss, in my opinion, especially during development.

The fix (Collections.synchronizedList) is intentionally conservative. A more granular approach using CopyOnWriteArrayListor ConcurrentHashMap could improve read-heavy throughput, but for a system where registrations are infrequent and correctness matters more than raw performance, the synchronized wrapper is clear and sufficient.

A second challenge was keeping the ServerProcessTest suite passing after the refactoring. That test class calls server.process(json) directly, bypassing the socket entirely. The

solution was to preserve the process() method on Server as a thin delegator to ClientHandler.process(), so the existing tests needed no changes. A new ClientHandlerTest class was also written to test the extracted handler logic in complete isolation in the Parking System.

Comparison of Threading Models

Three threading models are commonly used in Java server applications, and each has distinct trade-offs, which is something that I learned about greatly throughout the module 9 learning information this week.

Per-request threads (new Thread(handler).start() for every connection) are the simplest to understand. Each client gets a dedicated OS thread for its lifetime. The downside is that thread creation is not free. On a busy server receiving hundreds of requests per second, the constant allocation and teardown of threads wastes CPU time and memory. There is also no natural ceiling: under a request surge the JVM may create thousands of threads, exhausting heap space or OS resources.

Fixed thread pools (Executors.newFixedThreadPool(N)) amortize creation cost by reusing a bounded set of worker threads. The pool size N acts as a back-pressure mechanism: if all workers are busy, new tasks queue rather than spinning up new threads. This is the model used here. The trade-off is choosing N: too small and clients queue unnecessarily; too large and the benefit of bounding resources is lost. For the DU Parking system, where each request completes in well under a millisecond, a pool of 10 threads can serve thousands of requests per second.

Cached thread pools (Executors.newCachedThreadPool()) create threads on demand and reuse idle ones, with no fixed ceiling. They are effective for bursty workloads with short tasks but dangerous under sustained high load because the pool size is unbounded.

Virtual threads (Java 21+, Thread.ofVirtual()) represent a newer model where the JVM multiplexes many lightweight virtual threads onto a small number of OS threads. They eliminate the per-thread memory overhead almost entirely and are particularly well-suited to I/O-bound work like socket handling. The parking system could be migrated to virtual threads by replacing Executors.newFixedThreadPool with Executors.newVirtualThreadPerTaskExecutor() with minimal code change.

Reflection

This assignment was interesting to me because it made concrete a concept that is easy to understand abstractly, but overall harder to reason about in practice. That correctness in multithreaded code requires explicit thought about every shared resource, not just the obviously "sensitive" ones. The ArrayList inside ParkingOffice looked harmless until the moment two threads could write to it simultaneously.

From a design perspective, extracting ClientHandler as a separate Runnable class turned out to be more than a mechanical refactoring step. It clarified the responsibilities of the server: the Server class now does exactly one thing (accept connections and dispatch them), while ClientHandler does exactly one other thing (process a single request). This separation made the

unit test for ClientHandler straightforward to write (the socket is never needed for testing the logic) and it makes the code far easier to reason about when debugging concurrency issues. This assignment reinforced to me that clean separation of concerns and thread safety are not orthogonal goals; a well-decomposed class is usually easier to make thread-safe precisely because its responsibilities are narrow.